

Memorization Diagnostics for Code LLMs Should be Scale-Aware

Prateek Kumar Rajput^{1*}, Abdoul Aziz Bonkougou¹,
 Alberick Euraste Djiré¹, Xunzhu Tang¹, Yewei Song¹,
 Iyiola Emmanuel Olatunji¹, El Hacem Diallo¹, Jacques Klein¹,
 Tegawendé F. Bissyandé¹

^{1*}University of Luxembourg, Esch-sur-Alzette, Luxembourg.

*Corresponding author(s). E-mail(s): prateek.rajput@uni.lu;

Contributing authors: abdoul.bonkougou@uni.lu; euraste.djire@uni.lu;
xunzhu.tang@uni.lu; yewei.song@uni.lu; emmanuel.olatunji@uni.lu;
el-hacen.diallo@uni.lu; jacques.klein@uni.lu;
tegawende.bissyande@uni.lu;

Abstract

The extent to which large language models for code rely on memorization over genuine understanding remains highly debated. While current literature frequently reports widespread memorization, evaluating the underlying probing techniques across dense architectures reveals a severe breakdown in their utility at scale. Traditional encoder-style probes using perturbations such as synonym fuzzing or dead-code insertion struggle to expose memorization in scaled models, even on known-contaminated benchmarks, and decoder-style probes that rely on log probabilities show similar performance degradation. The specific mode of failure for these probes, particularly why such techniques disrupt smaller models but fail to impact larger ones, motivates us to untangle representation load from memorization rather than treating them as a single phenomenon. By applying invertible mathematical transforms to numeric problems, we isolate these two factors and reveal that scaled encoders successfully absorb substantial representation load while still converging on the correct family of solutions. In practical software engineering, this ability to adapt to varying surface forms is what truly matters for usability and generalizability in LLM and agentic applications. Whether a specific solution was seen during training becomes a much less pressing question because although memorization inflates scores on contaminated benchmarks, factoring out representation load makes it debatable how much we should truly care if a functional answer was originally memorized. Future evaluations

must therefore be built around separating these phenomena rather than relying on methodologies that quietly entangle them. Code and Data are available.*

Keywords: Code generation, LLM evaluation, memorization, contamination, representation load, robustness, metamorphic testing

1 Introduction

LLMs are rapidly becoming the default engine for automating software engineering tasks, and code synthesis sits at the frontier of this transformation. They achieve impressive benchmark accuracy [1–4], yet a persistent question shadows their progress. *Are they solving problems or recognizing them?* Recent work sharpens the worry, with analyses of agentic coding benchmarks arguing that some leading results lean on recall more than reasoning [5]. The dominant strategy in the literature for probing this question is to apply semantics-preserving perturbations to the prompt, such as renaming variables [6, 7], paraphrasing docstrings [8, 9], and inserting dead code [6, 7], then reading the resulting pass@ k drop as evidence bearing on memorization [10–12]. These *encoder-side* diagnostics operate in lexical and semantic space. They are intuitive and have no doubt enhanced our understanding, but they are also just convenient. Most studies say little about the extra demands these prompt changes place on the model, rarely examine scaled models where encoder capacity may absorb the perturbation entirely, and offer no formal guarantee that the rewritten prompt preserves the original task semantics. A complementary line of work targets the *decoder*. Methods such as CoDeC [13] measure how in-context examples shift token log-likelihoods to distinguish seen from unseen training data, but these probes are also often validated on small model families and do not address whether their discriminative signal survives as the model’s internal representation space scales and the probability mass spreads across a vastly larger set of plausible continuations. Other such decoder-style works include permutation tests on dataset ordering [14], multiple-choice contamination quizzes [15], and token-level likelihood analyses for verbatim code regurgitation [16].

We put the assumption that *the probe keeps working as models grow* under direct test, and on benchmarks with known contamination [2, 13, 17, 18], the encoder-side and decoder-side signals we studied faded as model capacity increased. This probe saturation pushed us toward a different question. Rather than asking whether a solution was seen during training, we ask *how faithfully a model carries its algorithmic knowledge through an unfamiliar way of stating the same task*. We treat this capacity as distinct from memorization and term it *representational load*, and our experiments later confirm that the two are separable. The framing borrows from metamorphic testing, where a known relation between inputs constrains the relation between outputs. To add representational load while holding the core task fixed, we leverage invertible numerical transforms. We develop both ideas below.

*https://github.com/pkrajput/isomorphic_prompts,
19248561

<https://doi.org/10.5281/zenodo.19248561>

Representational load. Representational load arises wherever the interface departs from training-like surface form. Domain-specific encodings, legacy data formats, unfamiliar API contracts, and project-specific conventions all impose interface conditions that no benchmark captures. On the encoder side, it requires the model to parse auxiliary constraints alongside the core task specification, mapping a richer, less familiar prompt to the correct underlying semantic representation. On the decoder side, it requires the model to honour those constraints token by token during autoregressive generation, interleaving novel formatting logic with the algorithmic solution it has internalized. A model that can only produce correct code under training-like surface conditions offers weaker guarantees than one that can transport its algorithmic knowledge through an unfamiliar representational channel. Building on this view, we first study one representative probe from each side of the literature, encoder and decoder, along a model-scale axis, and then propose a new perspective that leverages a property unique to software, the latent structure of code itself.

I/O isomorphisms. To probe behaviour beyond surface text manipulation, and to keep the task *algorithmically fixed* under a mathematical guarantee, we use *I/O isomorphisms*. Methodologically this is a form of metamorphic and robustness testing [19, 20] adapted to code LLMs. We apply a bijective value transform to the test-case integers and append an explicit encode/decode contract to the prompt, wrapping the interface in a representational layer that demands explicit computation without altering the underlying algorithmic task ($f(x) = y$). The metamorphic relation between the original test oracle and its isomorphic counterpart then serves as our oracle, so any deviation on a previously passing problem is, by construction, a representational-robustness failure rather than a change in the task itself. *The model cannot memorize its way through this layer.* It must decode every input, run the algorithm, and re-encode every output. For our main experiments we instantiate the transform as an affine integer isomorphism,

$$T_{\theta}(t) = at + b, \quad a \neq 0, \quad T_{\theta}^{-1}(t') = \frac{t' - b}{a},$$

and we additionally ablate over base-conversion and cubic bijection families. Here, t represents the original test-case integer input, while a and b are the randomly selected constants (with $a \neq 0$) that define the linear transformation parameterizing the affine isomorphism.

Our study yields three findings.

- Synonym substitution in prompts and token log-likelihood probes that rely on in-context examples both saturate at scale, losing their power to separate seen from unseen problems.
- Representational load degrades performance even in scaled models, yet this drop does not by itself reveal memorization.
- Opcode analysis shows that scaled models still reach the same solution families under added representational load, only less often, while smaller models drop whole families of solutions altogether.

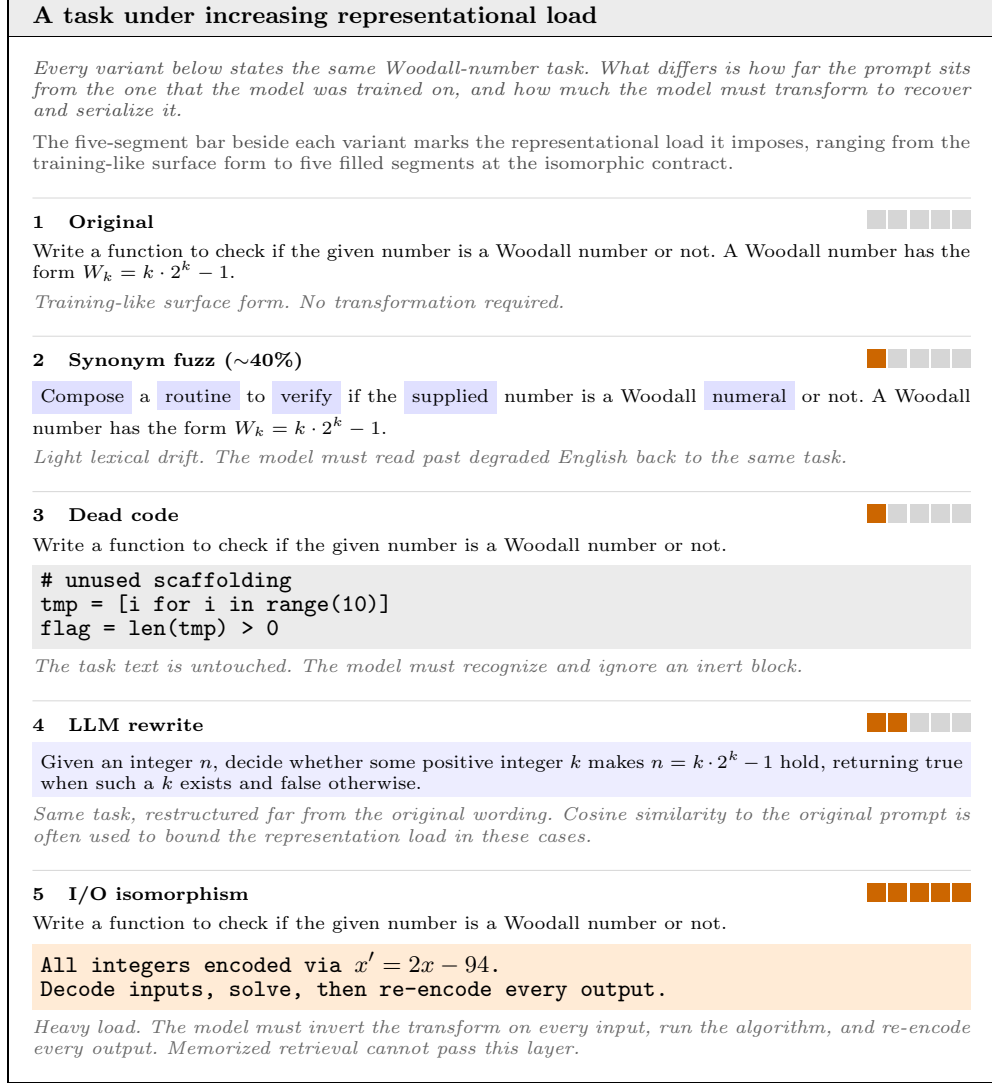


Fig. 1 The same task posed at increasing representational load, with the five-segment bar beside each variant indicating how much load it imposes. Synonym fuzzing, dead code, and a rewrite perturb the surface form only, while the isomorphic contract forces the model to invert a value transform, run the algorithm, and re-encode every output.

Of the three, the third is the central phenomenon of this work, and Figure 1 builds the intuition behind it. A single Woodall task is restated at growing distance from its training-like form, moving from light synonym and dead-code edits through a free-form rewrite to a full isomorphic contract, with each variant forcing the model to do more work to recover the task and then serialize its answer. The $\text{pass}@k$ drop that follows invites a naive reading in which the model has simply lost the algorithm, and we argue this reading is incomplete. What the added load strains is the route from a

MBPP Task 18 under a full isomorphic contract ($T_\theta(t) = 2t - 94$)			
Failure examples from 5 generations of the same I/O contract.			
Prompt and encoding contract Write a function to check if the given number is a Woodall number or not. A Woodall number has the form $W_k = k \cdot 2^k - 1$. All integers encoded via $x' = 2x - 94$. Decode inputs as $x = (x' + 94)/2$, solve, then re-encode every output. E.g. input -74 represents 10 ; if the answer is 0 , output -94 .			
Test cases: Original <code>is_woodall(383)==True is_woodall(254)==False is_woodall(200)==False</code> Iso <code>is_woodall(672)==True is_woodall(414)==False is_woodall(306)==False</code>			
Passing solution	3/5	Failing solution	2/5
<pre> def is_woodall(n_enc): n = (n_enc + 94) / 2 if n <= 0 or not n.is_integer(): return False n = int(n) k = 1 while True: w = k * (2**k) - 1 if w == n: return True if w > n: return False k += 1 </pre>			
<pre> def is_woodall(enc_n): n = (enc_n + 94) / 2 if not n.is_integer(): return (0*2)-94 == -94 n = int(n) if n <= 0: return (0*2)-94 == -94 k = 1 while True: w = k * (2**k) - 1 if w == n: return (1*2)-94 == -92 if w > n: return (0*2)-94 == -94 k += 1 </pre>			
<i>Why it fails.</i> The algorithmic loop is identical in both runs. The break is decoder-side: complying with the integer contract token-by-token, the model routes every boolean answer through an encoded arithmetic expression $((b*2)-94 == \dots)$ which in Python evaluates back to <i>True</i> , so all four returns collapse to the same value. The load of following the contract during generation, not the algorithm, is what fails, and this is the load our RQs target.			

Fig. 2 Example: MBPP Task 18 under ISO contract. The failure mode is far from memorization, under heavy representational load.

known solution to a correctly encoded one, not the model’s grasp of the solution itself. Section 2 makes this concrete on a single failing trace (Figure 2), where the algorithmic core is identical to that of a passing run and only the output encoding breaks.

2 Running Example

Figure 2 traces one MBPP task under the isomorphic contract. Across both the passing and failing generations, the `while` loop that enumerates Woodall candidates

$k \cdot 2^k - 1$ is *identical*. What separates a pass from a failure is the return statement. The failing generation pushes its boolean answer through the integer contract, writing comparisons such as `(0 * 2) - 94 == -94`. In Python, that expression is `True` on every branch, so the function reports every input as a Woodall number. The model has not lost the algorithm. It has confused the boundary between the encode-decode contract and Python’s own type system, and it cannot reliably serialize `False` once the native return type is boolean.

Narrow channels. This is the behaviour we name a *narrow channel*. The path that carries a model’s algorithmic knowledge from specification to code stays open under representational load, but it tightens, so the same solution families still appear, only less often. A degraded `pass@k` then measures a constricted route rather than a forgotten algorithm, and the Woodall trace is one instance of it, with the correct output surviving in three of five samples, while the loop never changes. Smaller models behave differently. Their channel closes rather than narrows, and entire solution families drop out, which is the collapse we separate from the graceful narrowing seen at scale.

The same trace also tells the effect apart from memorization. A retrieved training solution should shatter under the isomorphic prompt, whose transformed I/O does not resemble any plausible training instance, yet the scaled models keep the task intact and break only at the contract. Prior probes would log this is nothing more than a lower `pass@k` [10–12]. To recover the algorithmic core that `pass@k` hides, we turn to opcode divergence between solutions under the original and isomorphic prompts, measured over passing solutions and overall generations.

3 Related Work

3.1 Memorization and Contamination in Code

That memorization is real and measurable in large language models is by now well documented. Training data can be extracted near-verbatim from large models [21], and deduplicating the corpus reduces it while improving downstream behaviour [22]. For code in particular, contaminated benchmarks have been tied to inflated pass rates [12], models score higher on problems released before their training cutoff than on fresh ones [23], and the leakage of evaluation sets into training has proven both common and easy to overlook [24, 25]. The concern extends to the agentic setting, where recent analysis argues that some leading SWE benchmarks reflect recall rather than reasoning [5].

3.2 Probes targeting the Encoder and Decoder

On the encoder, the standard probe perturbs the prompt while preserving its meaning, then reads the `pass@k` drop as a memorization signal, through variable renaming [6, 7], docstring paraphrase [8, 9], or dead-code insertion [6, 7]. As we argue in Section 1, these manipulations stay in lexical and semantic space, rarely control for the extra parsing they impose, and carry no guarantee that the rewritten prompt preserves the original task. A complementary line attacks the question on the decoder. Membership inference asks whether a datum was seen, inferring it from the lowest-probability tokens of a

sequence [26] or from how a model scores a text against close neighbours [27], while counterfactual memorization [28] and influence functions [29] attribute behaviour to individual training examples. These methods sharpen what can be said about whether a specific input was seen, yet they answer a membership question rather than the capability question a practitioner cares about, which we try to target via separating the representational load.

3.3 Probe Reliability at Scale

Memorization is known to grow with model size and data duplication [21] and to emerge in partly predictable, scale-dependent ways [30], so a probe calibrated on a small family is reading a moving quantity. Membership-advantage probes make this concrete, having been validated on a handful of small, open-code models where they separate likely-seen inputs from unseen ones [10, 11], though whether that separation survives at the scales now deployed in practice remains untested. Training dynamics complicate the picture further. Grokking shows that models can fit their data long before they generalize, with generalization arriving only after extended training uncovers latent structure [31], an effect later traced to the gradual formation of structured internal circuits [32] and explained as a shift toward the most efficient representation [33], while memorization itself accrues across training in ways that do not track simple overfitting [34]. The duration and composition of training, not model size alone, may therefore be a decisive and largely uncontrolled variable, since two models of the same size can sit on opposite sides of a grokking-like transition.

3.4 Moving Targets and Our Perspective

The community’s main response to contamination has been to build moving targets, whether contamination-free live benchmarks [35], harder test suites that expose brittle pass rates [36], or benchmarks that evolve known problems into fresh variants [37]. We take a different route. Because capability rises predictably with scale [38] and larger models adapt to unfamiliar prompt specifications through in-context learning [39, 40], we hold the algorithm fixed and change only how it is stated, asking what survives once representational load is separated from recall rather than chasing ever-cleaner data.

4 Research Questions

What remains is to turn this intuition into a structured investigation, and we proceed by elimination. If the field’s standard probes still separated seen problems from unseen ones at frontier scale, the prevailing memorization reading would hold and little would need revising. We therefore put those probes to the test first, one on each side of the model, and treat their continued discriminative power as the hypothesis to be disproved. Only once that reading is shown to break do we introduce a load the model cannot have encountered in training, and ask what the resulting failure really is, whether the model has lost the algorithm or only the route that serializes it. The two leave the same trace in $\text{pass}@k$, so nothing short of this ordering can tell them apart.

Three research questions follow that path, two that close out the inherited account and one that opens the one we propose.

RQ1: To what extent do encoder-side memorization probes work at scale?

Synonym substitution and dead-code insertion stress lexical surface form. We test whether the resulting pass@ k drop still tracks any diagnostic signal in frontier models, or whether scaled encoders absorb the perturbation.

RQ2: To what extent do decoder-side memorization probes work at scale?

Likelihood-based probes such as CoDeC compare token-level scores with and without in-context examples. We test whether the seen-vs-unseen gap they rely on survives as the model scale grows.

RQ3: To what extent can representational load break performance? Using I/O isomorphisms, we keep the algorithmic task provably fixed and only change the interface. If pass@ k still drops, the failure cannot be a memorization artifact and must come from the load itself.

5 Methodology

This section describes the experimental setup common to all three diagnostic axes, covering the models and their selection rationale (Section 5.1), the datasets and evaluation conditions (Section 5.2), and the protocols specific to each axis (Sections 5.3–5.5). An overview of the full methodology can be seen in Figure 3.

5.1 Models

Table 1 Models used in our experiments. **(a)** Encoder-side lexical probing and I/O isomorphism experiments. **(b)** Decoder-side experiments (CoDeC).

(a)			(b)		
Model	Params	Context	Model	Params	Context
<i>Open-weight</i>			<i>Open-weight</i>		
deepseek-coder-6.7b-instruct	6.7B	16K	Pythia-410M	0.4B	2K
Llama-3.1-8B-Instruct	8B	128K	Pythia-1.4B	1.4B	2K
CodeLlama-13B-Instruct	13B	16K	Pythia-12B	12B	2K
StarCoder2-15B	15B	16K	Llama-3.1-70B	70B	128K
Codestral-22B-v0.1	22B	32K	Nemotron-4-340B-Instruct	340B	4K
Qwen2.5-Coder-32B-Instruct	32B	128K	Llama-3.1-405B-Instruct	405B	128K
Llama-3.1-70B-Instruct	70B	128K	<i>Closed-source</i>		
<i>Closed-source</i>			davinci-002	—	16K
gpt-4o-mini-2024-07-18	—	128K			
gpt-4o-2024-08-06	—	128K			
gemini-2.0-flash	—	1M			
gemini-2.5-flash	—	1M			

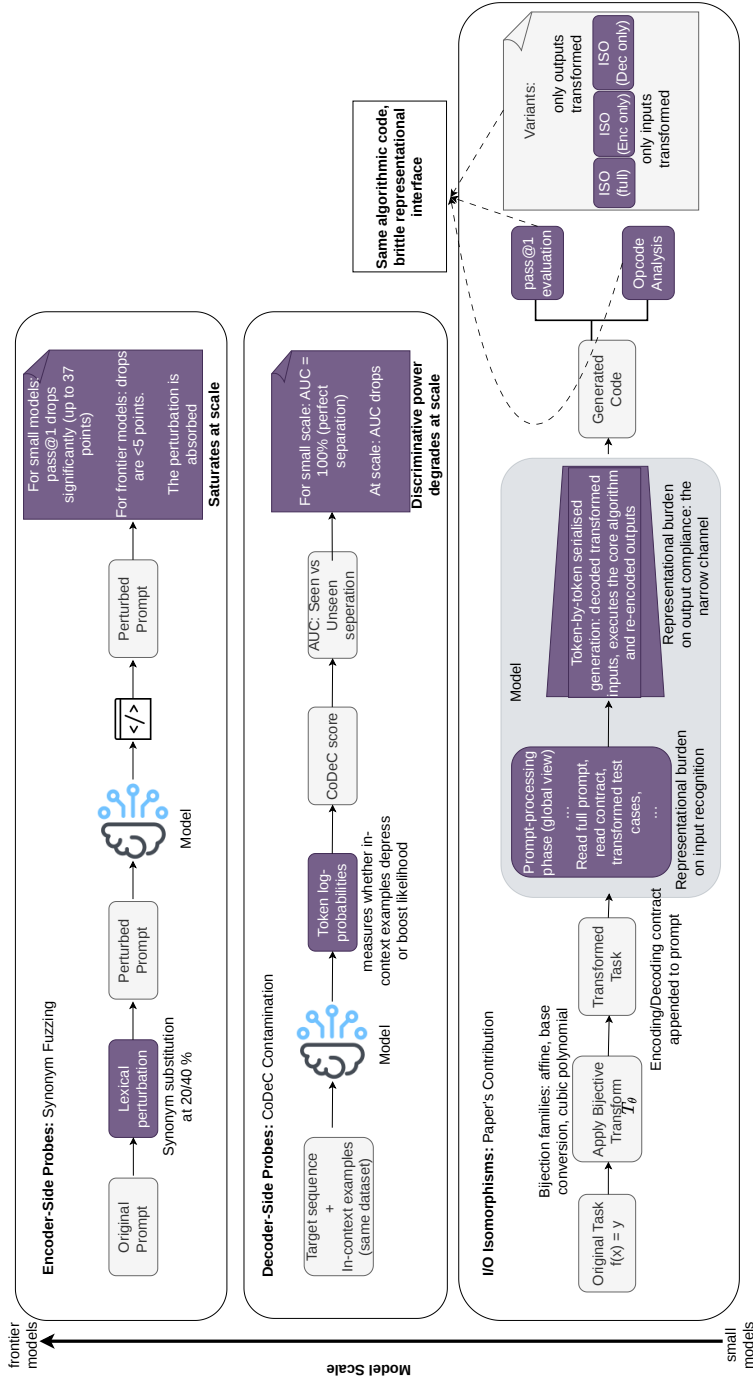


Fig. 3 Overview of diagnostic protocols. We evaluate three axes across model scale, namely encoder-side synonym fuzzing (top), decoder-side CoDeC contamination probing (middle), and our proposed I/O isomorphism protocol with opcode analysis (bottom). The isomorphism preserves the algorithmic task exactly ($f(x)=y \iff T_\theta(f(T_\theta^{-1}(x)))=y'$) while isolating representational load on both the prompt-processing and generation phases.

We select 11 models spanning a wide range of capabilities for the encoder-side and isomorphism experiments (Table 1a), prioritizing models commonly used for coding tasks and covering a broad parameter-count range. For the decoder-side contamination analysis (Section 5.3), we construct a separate scale axis of 7 models (Table 1b) that permit *teacher-forced scoring*, computing per-token log-probabilities of a target sequence conditioned on its ground-truth prefix, without generating new tokens. For open-weight models (Pythia 410M, 1.4B, 12B; Nemotron-4-340B; Llama-3.1-405B), we use the vLLM library and access prompt-token log-probabilities directly through `prompt_logprobs`. For davinci-002, we use OpenAI’s legacy completions endpoint, which exposes equivalent functionality via `echo=True` with `logprobs`, to include at least one commercial model in our study.

A deliberate design choice guides our model selection. All models in our scale axis are dense architectures, not mixture-of-experts (MoE). Because MoE models activate only a fraction of their parameters per token, the relationship between nominal parameter count and effective capacity is ambiguous. A 340B MoE model may behave more like a much smaller dense model on any given input. Since our analysis aims to study how memorization and generalization behaviours change as a function of scale, dense architectures provide a cleaner signal, as total parameters directly reflect the capacity available during inference. How decoder-side diagnostics should be adapted for MoE architectures, where routing decisions may interact with memorization patterns in ways that dense models do not exhibit, remains an open question.

5.2 Datasets, Conditions, and Generation Setup

Table 2 Evaluation datasets across the study’s two diagnostic axes. The upper block covers the I/O isomorphism and encoder-side perturbations. The lower block covers the CoDeC probe, split into corpora that postdate every model’s training cutoff (*Unseen*) and corpora that are documented or probable members of the training mixtures (*Seen*).

Dataset	Tasks / Role	Domain	Released
<i>I/O isomorphism and encoder-side perturbations</i>			
MBPP [2]	257	Mostly-basic Python	Aug 2021
EffiBench [41]	338	Efficiency	Nov 2024
BigOBench [42]	640	Algorithmic complexity	Mar 2025
<i>Decoder-side perturbations (CoDeC)</i>			
gpqa_diamond [43]	Unseen	Graduate-level Q&A	Nov 2023
livecodebench.v5 [35]	Unseen	Competitive programming	Jan 2025
wikipedia (Pile) [17]	Seen	Encyclopedic text	Dec 2020
hackernews (Pile) [17]	Seen	Web forum text	Dec 2020
cc.2024_high [18]	Seen [†]	Common-Crawl web text	2024

[†] Substituted for `hackernews` when probing Nemotron-4-340B-Instruct, following the corpus-overlap designation of Zawalski et al. [13].

Datasets. We evaluate on three Python benchmarks spanning both a difficulty axis and a contamination axis (Table 2). MBPP sits at the easy end of the difficulty spectrum and is among the most widely used benchmarks in the code LLM literature [10, 12, 35]. Every model in our study was released after its publication, making all potentially contaminated on this benchmark. EffiBench and BigOBench, by contrast, were published in 2024 and 2025, respectively, after the training cutoff of all models except Gemini-2.5-Flash. This creates a natural experiment. If memorization were the primary driver of performance, we would expect systematically larger ISO drops on the contaminated benchmark (MBPP), while drops on unseen benchmarks (EffiBench and BigOBench) should not be systematically large. We test this prediction directly in Section 6. All three benchmarks use Python exclusively, removing cross-language variation as a confound.

Evaluation conditions. We utilize six conditions for inference. The first four probe representational invariance via I/O isomorphisms; the last two serve as encoder-side baselines operating in lexical space.

Synonym fuzzing protocol. For the SYN-20% and SYN-40% conditions, we replace each non-connector, non-code-token word in the natural-language portion of the prompt with a WordNet synonym at the specified probability. This preserves task semantics while altering surface form, the same axis targeted directly in recent work [10] and by other similar probes like variable renaming [6, 7] and paraphrasing [8, 9]. We work under the same hypothesis as theirs. If a model has merely memorized a solution, altering the surface form should degrade retrieval.

Generation setup. For each (task, condition) pair, we generate $n=5$ completions at temperature $T=0.0$. Fixing the temperature at zero removes temperature-driven sampling as a source of variation, so that the behaviour we observe under each condition is not confounded by deliberate exploration of the output distribution. Greedy decoding is nonetheless not bitwise deterministic in practice, because floating-point reductions on accelerators are non-associative and sensitive to batch composition, and the hosted endpoints we query give no determinism guarantee, so repeated generations of the same prompt still diverge wherever two continuations lie close in probability. *It is this residual nondeterminism, not temperature, that supplies the per-problem solution spread we analyse.* We generate five completions for two reasons. First, our opcode-entropy analysis (Section 5.5) reads off a per-problem distribution of solutions, and pooling opcode frequency vectors across five generations enables meaningful per-problem Jensen-Shannon divergence estimates between conditions. Second, both EffiBench and BigOBench admit multiple valid solutions to each problem, so several generations help to characterise the solution space the model explores. The same decoding regime is held fixed across the ORIGINAL and isomorphic conditions, so any divergence we report reflects the condition rather than a change in how solutions are drawn.

5.3 Decoder-Side Protocol (CoDeC)

We reproduce CoDeC [13], a contamination diagnostic that operates on output log-probabilities. Each dataset receives a contamination score $S_{\text{CoDeC}}(D) = (1/N) \sum_i \mathbf{1}[\Delta(x_i) < 0]$, where $\Delta(x_i)$ is the change in mean token log-likelihood of

target x_i when adding in-context samples from the same dataset. In-context examples typically boost confidence for unseen data but may *reduce* it when the dataset was memorized during training by disrupting learned retrieval patterns. The dataset-level AUC quantifies separation between seen and unseen datasets. Formally, with seen-score set S^+ and unseen-score set S^- ,

$$\text{AUC} = \frac{1}{|S^+||S^-|} \sum_{s^+ \in S^+} \sum_{s^- \in S^-} [\mathbf{1}(s^+ > s^-) + 0.5 \mathbf{1}(s^+ = s^-)].$$

To keep frontier-scale runs feasible, we use a constrained but consistent setup with two seen and two unseen datasets per run. We designate `gpqa_diamond` [43] and `livecodebench_v5` [35] as unseen, since both postdate the training cutoff of every model in our decoding experiments. For the seen datasets, we follow the designations reported by Zawalski et al. [13] and use `hackernews` and `wikipedia` from the pile dataset [17] (with `cc.2024.high` [18] replacing `hackernews` for Nemotron-4-340B-Instruct due to known corpus overlap).

5.4 I/O Isomorphism Protocol (Metamorphic Testing for Code LLMs)

A coding-benchmark instance defines a mapping $f : \mathcal{X} \rightarrow \mathcal{Y}$ over textual I/O representations. We define a family of invertible value transforms parameterized by θ :

$$T_\theta : \mathcal{V} \rightarrow \mathcal{V}, \quad T_\theta^{-1}(T_\theta(v)) = v.$$

For the main experiments, we use affine integer isomorphisms:

$$T_\theta(t) = at + b, \quad a \neq 0, \quad T_\theta^{-1}(t') = \frac{t' - b}{a}.$$

Every integer in the test I/O is mapped through T_θ . Original tests (x, y) become (x', y') with $x' = T_\theta(x)$, $y' = T_\theta(y)$. The prompt is augmented with a *contract* specifying the values of a and b , along with instructions to decode inputs, solve, and return encoded outputs. We additionally ablate with two alternative bijection families, base conversion and cubic polynomials, to test whether the observed brittleness generalizes beyond affine transforms (described in Section 7.1).

Correctness equivalence (the metamorphic relation). Because T_θ is bijective,

$$f(x) = y \iff T_\theta(f(T_\theta^{-1}(x'))) = y'.$$

Any accuracy difference between ORIGINAL and Isomorphic variant (ISO) of a problem is therefore attributable to representational handling, not to a change in the underlying task.

5.5 Opcode Entropy as an Algorithmic Proxy

Pass@ k measures whether the model produces *any* correct solution but says nothing about *what kind*. To characterize the solution space beyond binary correctness, we compile each generated Python program to bytecode using `dis` and compute the Shannon entropy of its opcode-frequency distribution,

$$H(\pi) = - \sum_{o \in \mathcal{O}} \pi(o) \log_2 \pi(o),$$

where $\pi(o)$ is the normalized frequency of opcode o . Programs relying on similar algorithmic strategies produce similar opcode profiles, making entropy a lightweight proxy for the algorithmic family a solution belongs to. To quantify distributional divergence at the per-problem level, we pool the opcode frequency vectors from all 5 generations for a given (problem, condition) pair and compute the Jensen-Shannon divergence (JSD) against the corresponding pooled distribution under ORIGINAL. JSD is bounded in $[0, 1]$ (base 2) and symmetric, providing a principled measure of how far a model’s solution strategy shifts under perturbation.

6 Results

Across all eleven models and three benchmarks, lexical probes (Syn) and likelihood probes (CoDeC) both saturate at scale. Our metamorphic ISO protocol drops frontier-model pass@1 by 14–30 points even though the algorithmic task is provably unchanged, yet these are not algorithmic failures. Pooled-opcode JSD between ORIGINAL and ISO generations stays near zero for scaled models, which keep the same control flow and merely mis-serialize the interface, and spikes only for smaller models on harder benchmarks, which abandon the strategy outright.

6.1 Encoder-Side Probes Saturate at Scale

Table 3 reports pass@1 under SYN-20 and SYN-40 across all eleven models and three benchmarks. The results show a clear saturation pattern. Synonym fuzzing remains diagnostic for small and some mid-tier models, but is nearly inert at the frontier.

Frontier models absorb synonym perturbation. For frontier models, pass@1 drops remain small across benchmarks and conditions. Gemini-2.0-Flash loses at most 5.4 points, while Gemini-2.5-Flash loses even less, with a maximum drop of 2.1 points and occasional gains on BigOBench and EffiBench. GPT-4o shows the same overall pattern, with modest and consistent losses. Because these perturbations follow the synonym-fuzzing protocol of Djiré et al. [10], the result points to failure of lexical perturbations as a tool to study memorization, which was validated on smaller models but loses discriminative power at the frontier scale, where encoders readily collapse semantically equivalent surface forms.

Mid-tier models show thresholded degradation. GPT-4o-mini, Llama-3.1-70B, and Qwen2.5-Coder-32B fall in an intermediate regime. Under Syn-20, drops are modest, but Syn-40 produces larger and benchmark-dependent losses. For example, Llama-3.1-70B drops 7.9 points on MBPP under Syn-20 but 27.3 under Syn-40,

Table 3 Pass@1 (%) across all conditions and models. Values in parentheses show the relative change from ORIGINAL. $\pm$ denotes the half-width of a 95 % bootstrap CI (10000 resamples over problems). Tier labels: **F** = frontier, **M** = mid-tier, **S** = small.

Dataset	Model	Orig	Iso	Iso (Enc)	Iso (Dec)	Syn-20	Syn-40
BigO	gemini-2.5-flash ^F	43.2 $\pm$ 2.2	25.5 $\pm$ 4.1 (-41%)	35.4 (-18%)	30.1 (-30%)	45.1 (+4%)	46.2 (+7%)
	gemini-2.0-flash ^F	65.9 $\pm$ 2.0	50.3 $\pm$ 4.9 (-24%)	55.4 (-16%)	54.8 (-17%)	62.2 (-6%)	60.5 (-8%)
	gpt-4o-2024-08-06 ^F	50.9 $\pm$ 2.1	37.2 $\pm$ 3.1 (-27%)	46.5 (-9%)	40.1 (-21%)	49.4 (-3%)	48.0 (-6%)
	gpt-4o-mini-2024-07-18 ^M	41.5 $\pm$ 2.0	24.9 $\pm$ 4.4 (-40%)	33.8 (-19%)	27.5 (-34%)	37.4 (-10%)	33.2 (-20%)
	Llama-3.1-70B-Instruct ^M	42.6 $\pm$ 2.0	20.9 $\pm$ 4.7 (-51%)	32.4 (-24%)	27.3 (-36%)	37.5 (-12%)	31.1 (-27%)
	Qwen2.5-Coder-32B-Instruct ^M	30.1 $\pm$ 1.9	18.7 $\pm$ 2.8 (-38%)	24.4 (-19%)	22.3 (-26%)	27.7 (-8%)	26.8 (-11%)
	Codestral-22B-v0.1 ^S	22.0 $\pm$ 1.7	12.6 $\pm$ 3.9 (-43%)	17.8 (-19%)	13.9 (-37%)	17.2 (-22%)	16.1 (-27%)
	StarCoder2-15B ^S	0.6 $\pm$ 0.4	0.0 $\pm$ 0.6 (-100%)	0.3 (-50%)	0.0 (-100%)	0.3 (-50%)	0.0 (-100%)
	CodeLlama-13B-Instruct ^S	2.1 $\pm$ 0.6	1.7 $\pm$ 1.5 (-19%)	1.9 (-10%)	1.8 (-14%)	1.2 (-43%)	0.8 (-62%)
	Llama-3.1-8B-Instruct ^S	11.2 $\pm$ 1.3	3.8 $\pm$ 2.9 (-66%)	7.4 (-34%)	4.7 (-58%)	8.9 (-21%)	2.5 (-78%)
Effi	deepseek-coder-6.7b-instruct ^S	14.5 $\pm$ 1.5	5.1 $\pm$ 3.3 (-65%)	10.2 (-30%)	6.5 (-55%)	8.3 (-43%)	4.8 (-67%)
	gemini-2.5-flash ^F	26.2 $\pm$ 3.1	11.7 $\pm$ 5.1 (-55%)	21.0 (-20%)	15.4 (-41%)	27.0 (+3%)	28.1 (+7%)
	gemini-2.0-flash ^F	54.4 $\pm$ 3.1	29.3 $\pm$ 5.8 (-46%)	45.2 (-17%)	35.0 (-36%)	53.2 (-2%)	51.8 (-5%)
	gpt-4o-2024-08-06 ^F	62.2 $\pm$ 3.3	33.8 $\pm$ 6.2 (-46%)	52.1 (-16%)	40.2 (-35%)	60.5 (-3%)	58.4 (-6%)
	gpt-4o-mini-2024-07-18 ^M	48.0 $\pm$ 3.6	22.6 $\pm$ 5.3 (-53%)	36.5 (-24%)	26.4 (-45%)	43.7 (-9%)	39.4 (-18%)
	Llama-3.1-70B-Instruct ^M	36.2 $\pm$ 3.2	13.8 $\pm$ 5.0 (-62%)	23.5 (-35%)	17.7 (-51%)	32.2 (-11%)	24.6 (-32%)
	Qwen2.5-Coder-32B-Instruct ^M	39.5 $\pm$ 3.3	20.1 $\pm$ 4.0 (-49%)	30.8 (-22%)	27.3 (-31%)	35.9 (-9%)	28.4 (-28%)
	Codestral-22B-v0.1 ^S	21.3 $\pm$ 2.7	10.7 $\pm$ 4.4 (-50%)	16.2 (-24%)	12.1 (-43%)	18.9 (-11%)	7.6 (-64%)
	StarCoder2-15B ^S	3.9 $\pm$ 1.3	0.0 $\pm$ 3.1 (-100%)	1.2 (-69%)	0.3 (-92%)	0.3 (-92%)	0.6 (-85%)
	CodeLlama-13B-Instruct ^S	0.4 $\pm$ 0.4	0.0 $\pm$ 0.9 (-100%)	0.2 (-50%)	0.0 (-100%)	0.9 (+125%)	0.0 (-100%)
MBPP	Llama-3.1-8B-Instruct ^S	14.2 $\pm$ 2.2	3.7 $\pm$ 3.7 (-74%)	8.8 (-38%)	5.0 (-65%)	11.4 (-20%)	9.1 (-36%)
	deepseek-coder-6.7b-instruct ^S	9.6 $\pm$ 2.0	2.9 $\pm$ 3.1 (-70%)	6.4 (-33%)	3.8 (-60%)	7.8 (-19%)	6.2 (-35%)
	gemini-2.5-flash ^F	77.4 $\pm$ 2.0	47.2 $\pm$ 3.2 (-39%)	66.6 (-14%)	54.8 (-29%)	75.3 (-3%)	76.1 (-2%)
	gemini-2.0-flash ^F	80.8 $\pm$ 1.8	58.1 $\pm$ 4.1 (-28%)	71.1 (-12%)	59.0 (-27%)	79.2 (-2%)	76.4 (-5%)
	gpt-4o-2024-08-06 ^F	84.2 $\pm$ 1.8	58.5 $\pm$ 3.7 (-31%)	76.8 (-9%)	62.3 (-26%)	79.1 (-6%)	76.6 (-9%)
	gpt-4o-mini-2024-07-18 ^M	73.0 $\pm$ 2.1	43.8 $\pm$ 3.2 (-40%)	61.3 (-16%)	48.9 (-33%)	68.3 (-6%)	62.0 (-15%)
	Llama-3.1-70B-Instruct ^M	71.8 $\pm$ 2.1	41.4 $\pm$ 3.0 (-42%)	59.6 (-17%)	47.0 (-35%)	63.9 (-11%)	44.5 (-38%)
	Qwen2.5-Coder-32B-Instruct ^M	84.0 $\pm$ 1.8	45.4 $\pm$ 2.1 (-46%)	68.0 (-19%)	51.2 (-39%)	76.4 (-9%)	52.9 (-37%)
	Codestral-22B-v0.1 ^S	72.0 $\pm$ 2.1	44.1 $\pm$ 2.8 (-39%)	59.4 (-18%)	48.2 (-33%)	64.2 (-11%)	35.1 (-51%)
	StarCoder2-15B ^S	24.3 $\pm$ 2.0	0.8 $\pm$ 3.1 (-97%)	13.6 (-44%)	4.1 (-83%)	6.5 (-73%)	6.4 (-74%)
	CodeLlama-13B-Instruct ^S	12.8 $\pm$ 1.5	2.5 $\pm$ 2.1 (-80%)	7.8 (-39%)	3.8 (-70%)	13.2 (+3%)	4.3 (-66%)
	Llama-3.1-8B-Instruct ^S	48.2 $\pm$ 2.4	14.5 $\pm$ 3.1 (-70%)	32.4 (-33%)	19.8 (-59%)	41.2 (-15%)	23.6 (-51%)
	deepseek-coder-6.7b-instruct ^S	52.1 $\pm$ 2.4	16.7 $\pm$ 3.6 (-68%)	36.0 (-31%)	22.4 (-57%)	44.8 (-14%)	30.2 (-42%)

while Qwen2.5-Coder-32B shows a similar jump on MBPP. This non-linearity suggests that mid-tier encoders tolerate moderate lexical noise but degrade once perturbation density crosses a threshold.

Small models degrade sharply, but the signal conflates fragility with memorization. Below 22B parameters, synonym fuzzing causes large drops across all benchmarks. Codestral-22B, for instance, loses 36.9 points on MBPP under Syn-40, 13.7 on EffiBench, and 5.9 on BigOBench, with similar trends for Llama-3.1-8B and deepseek-coder-6.7b. If synonym fuzzing primarily disrupted memorized retrieval, the largest effects should appear on MBPP, the only benchmark plausibly contaminated for all models, with smaller drops on EffiBench and BigOBench, which postdate their training cutoffs. The data does not support this. Relative losses on unseen benchmarks are often comparable or larger, but this is confounded by task difficulty that was untested in previous works. Small models already have low baseline accuracy on harder tasks, so even small absolute drops become large relative declines. The diagnostic, therefore, mixes encoder fragility with memorization and cannot cleanly separate the two.

RQ1. *To what extent do encoder-side memorization probes work at scale?*

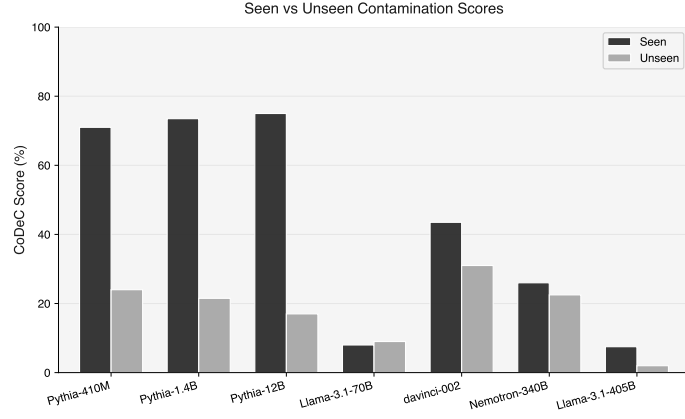
Their diagnostic value diminishes as scale increases. Synonym fuzzing at 20% and 40% rates, following the protocol of Djiré et al. [10], produces <8-point pass@1 drops for frontier models across all three benchmarks, while mid-tier models degrade moderately and small models collapse. The probe retains some signal at smaller scales but appears to lose much of its discriminative power at the frontier, at least for the models and benchmarks we examine.

6.2 CoDeC Loses Discriminative Power at Scale

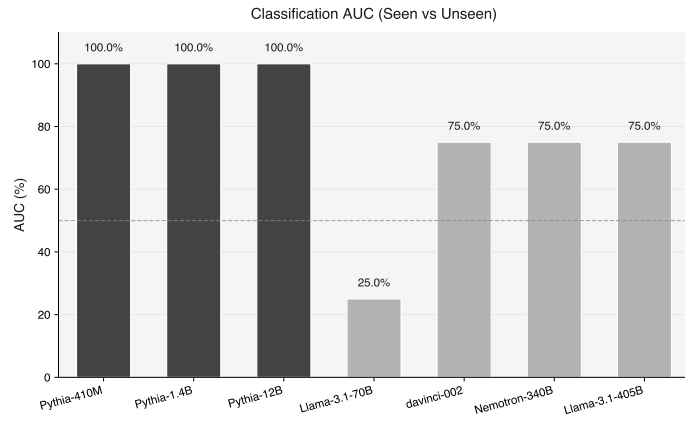
Table 4 CoDeC contamination scores across model scale. *Seen* and *Unseen* report the mean contamination score for datasets designated as in or absent from the training corpus. AUC quantifies the separation between the two.

Model	Params	Seen (%)	Unseen (%)	AUC (%)
Pythia-410M	0.4B	71.0	24.0	100.0
Pythia-1.4B	1.4B	73.5	21.5	100.0
Pythia-12B	12B	75.0	17.0	100.0
Llama-3.1-70B	70B	8.0	9.0	25.0
davinci-002	—	43.5	31.0	75.0
Nemotron-4-340B	340B	26.0	22.5	75.0
Llama-3.1-405B	405B	7.5	2.0	75.0

Table 4 and Figure 4 show CoDeC contamination scores across model scale. Our *unseen* labels (`gpqa_diamond`, `livecodebench_v5`) are guaranteed, since both datasets postdate the training cutoffs of all models in our decoder-side experiments. The *seen*



(a)



(b)

Fig. 4 (a) Mean CoDeC scores across 4 datasets designated as *Seen* or *Unseen* (Section 5.3). (b) AUC quantifying the separation between seen and unseen scores across 4 datasets.

labels (`hackernews`, `wikipedia`, and `cc_2024_high` for Nemotron) follow prior corpus-overlap analyses and documented training mixtures [13, 17, 44]. Although the seen status is therefore probable rather than certain, this asymmetry is shared across all models and does not affect the trend. Our reproduction matches the original CoDeC behaviour on Pythia: seen scores lie between 71–75%, unseen scores between 17–24%, and AUC remains 100% across all three checkpoints.

The gap reduces with scale. As model size increases, both contamination scores and seen-unseen separation shrink (Figure 4a). `davinci-002` narrows the gap to 12.5 points, Nemotron-4-340B to 3.5, and Llama-3.1-405B to 5.5. Llama-3.1-70B even inverts the expected ordering, yielding an AUC of 25%, though this is likely an anomaly given the small number of datasets. Even ignoring that inversion, seen-dataset scores drop from 71–75% at the Pythia scale to 7.5% at 405B, showing that in-context augmentation shifts token log-likelihoods much less in larger models.

Why does the signal degrade? CoDeC measures how in-context examples shift token log-likelihoods. Memorized targets should be disrupted, while novel targets should benefit. This assumes memorized content occupies a concentrated region of probability space that context can perturb. As models scale, that signal weakens for two reasons. First, probability mass is spread across a richer representation space, diluting concentrated retrieval effects. Second, larger models have stronger in-context learning abilities [39, 40], allowing them to use added context constructively for both seen and unseen targets, which reduces the asymmetry CoDeC relies on.

RQ2. *To what extent do decoder-side memorization probes work at scale?*

Their discriminative power appears to weaken with scale. CoDeC’s seen-unseen gap shrinks from 58 points (Pythia-12B) to 5.5 points (Llama-3.1-405B), and AUC drops from 100% to 75%. This trend is consistent with the dilution of localized retrieval signals in richer representation spaces and with the strengthening of in-context learning abilities that may override the memorization-specific asymmetry on which CoDeC depends.

6.3 Representational Load Degrades Performance Without Erasing the Algorithm

To answer RQ3 we break it into two smaller sub-questions that can each be tested on their own. A `pass@1` drop alone does not pin down the extent to which representational load breaks performance, because the same drop appears both when a model loses the algorithm under the unfamiliar interface and when it keeps the algorithm but mis-serializes the output. RQ3a isolates where in the model the degradation arises, and RQ3b establishes what kind of failure it is.

6.3.1 A Decoder-Side Bottleneck Drives the Degradation

Table 3 reports `pass@1` under all evaluation conditions. The full ISO contract drops frontier-model `pass@1` by 14–30 absolute points on a provably fixed task, which on its own shows that representational load is sufficient to cause substantial degradation.

To localize the source, we decompose the contract into ISO (ENC ONLY), which transforms only the inputs the model must decode, and ISO (DEC ONLY), which transforms only the outputs the model must encode. For Gemini-2.0-Flash on MBPP, decoding inputs alone (ISO (ENC ONLY)) costs a modest 9.7 points (80.8→71.1), whereas encoding outputs token by token (ISO (DEC ONLY)) costs a far larger 21.8 points (80.8→59.0), close to the 22.7-point penalty of the full ISO condition (80.8→58.1). This split holds across datasets and across scaled models. The bottleneck is output-side contract compliance rather than input recognition, consistent with autoregressive generation struggling more with formatting constraints held over many decoding steps than the encoder does with parsing a transformed prompt once.

RQ3a. *Where does the degradation from representational load originate?*

Primarily on the decoder side. I/O isomorphisms drop pass@1 by 14–30 absolute points in frontier models, a vulnerability that purely lexical probes such as synonym fuzzing fail to surface. Decomposing the contract shows that most of this penalty comes from output-side encoding rather than input-side decoding, pointing to contract compliance during generation as the dominant cost.

6.3.2 Opcode Analysis Separates Compliance from Competence

A pass@1 drop does not say whether the model has lost the algorithm or has kept it and only mis-serialized the interface. We read this distinction off the opcode-entropy analysis, treating a near-zero shift in the opcode distribution as evidence that the underlying strategy is preserved and a large shift as evidence that it is abandoned.

Scaled models preserve core algorithms and control flow across all representational loads. For frontier models such as Gemini-2.0-Flash and GPT-4o, the per-problem JSD over all solutions, not just passing ones, under ISO and ISO (DEC ONLY) stays near zero across all three benchmarks (Figure 6). The bytecode footprint under the metamorphic prompt is close to indistinguishable from that under the original prompt, and the model opts for the same loops, comparisons, and control flow on the exact problems where pass@1 has just dropped by 14–30 points. The combined distributions in Figure 5 track closely between ORIGINAL and ISO, with the model selecting the same algorithmic families and producing near-identical opcode profiles while fumbling the final output representation more often. As a calibration check, JSD under SYN-20 and SYN-40 also stays near zero for all models, which indicates that the metric flags preserved core logic rather than any prompt change.

Smaller models lose algorithmic stability as task complexity grows. The opcode distributions reveal a contrast between smaller and mid-tier models, with the latter heavily dependent on dataset difficulty. On easier datasets such as MBPP, models like Codestral-22B behave much like scaled models and keep JSD relatively low under ISO. On harder datasets such as BigOBench and EffiBench, however, smaller models buckle under the representational load and their JSD spikes (Figure 6). This suggests that when the underlying task already sits at the boundary of their capability, the added interface load pushes them to abandon the correct strategy and diverge

to qualitatively different and often incorrect solution families, or to fail to produce valid logic at all.

Scale expands algorithmic coverage, not just reliability. Comparing the ORIGINAL opcode profiles *across models* reveals a further dimension of scaling. On MBPP, Gemini-2.0-Flash and Codestral-22B produce overlapping entropy distributions, and the algorithmic repertoire required for basic tasks is within reach of both. On EffiBench and BigOBench the distributions diverge before any perturbation is applied (Figures 5c and 5d). Gemini-2.0-Flash spans two peaks and performs better at pass@1, while the smaller Codestral-22B largely skips the second peak, consistent with a family of solutions present in the larger model’s repertoire but absent from the smaller one.

RQ3b. *Is the degradation a competence failure or a compliance failure?*

A compliance failure for scaled models and a competence failure for smaller ones. Despite large pass@1 drops, frontier models show near-zero opcode JSD under isomorphism, which indicates they attempt the same algorithmic core and mainly fumble the representational channel. Smaller models, by contrast, abandon their strategies under the same load on complex tasks, a pattern more consistent with shallow and fragile task comprehension than with mere mis-serialization.

RQ3. *To what extent can representational load break performance?*

Sharply, but in scaled models without erasing the algorithm. The ISO contract drops frontier-model pass@1 by 14–30 absolute points on a provably fixed task, so the loss is the load rather than a memorization artifact. The penalty is concentrated on the decoder, where the model must serialize an encoded output token by token (RQ3a), and for scaled models the algorithmic core survives, surfacing as a narrowed rather than a closed route from specification to code (RQ3b). The competence failure that a bare pass@1 drop might suggest appears only in smaller models on tasks already near their capability limit.

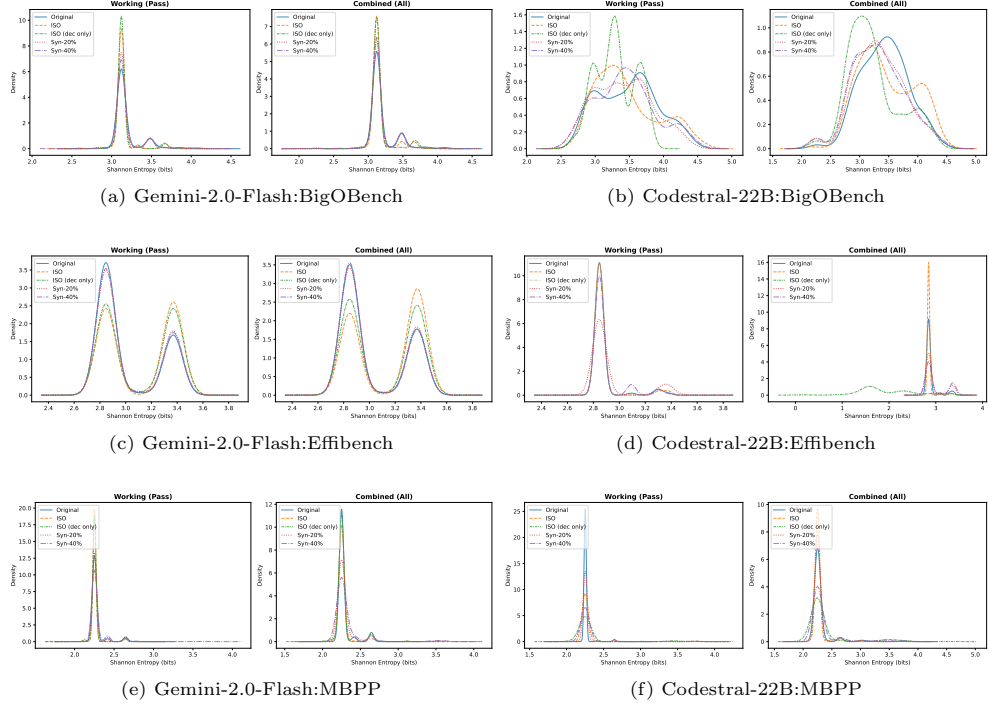
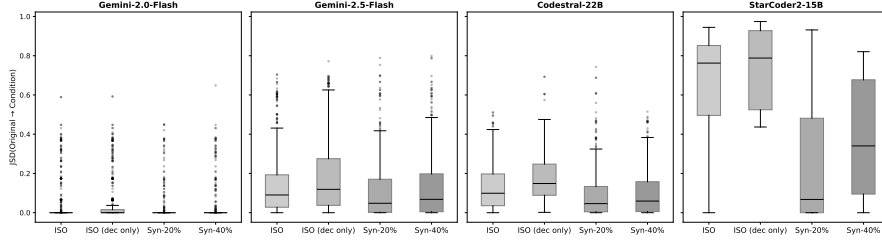
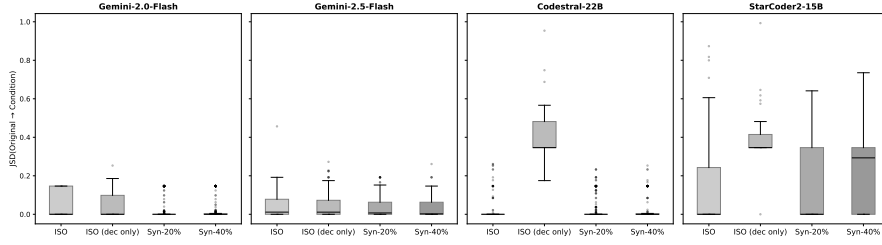


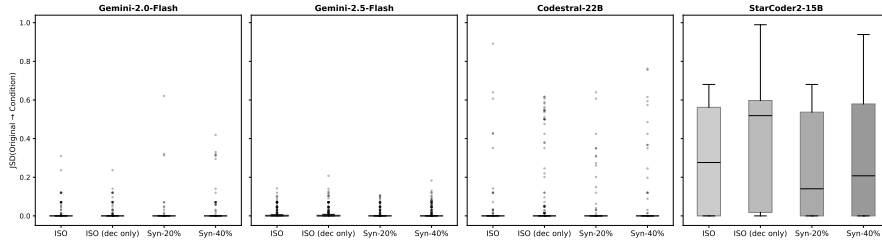
Fig. 5 Static opcode-entropy distributions (KDE) for selected model-dataset pairs. The left column shows Gemini-2.0-flash producing nearly identical entropy profiles under ORIGINAL and ISO, both for working solutions and for all generated code. The right column shows that Codestral-22b diverges substantially or produces no working solutions under ISO on harder datasets.



(a) BigOBench



(b) EffiBench



(c) MBPP

Fig. 6 Per-problem Jensen-Shannon divergence of pooled opcode distributions (CONDITION vs. ORIGINAL).

7 Ablation Studies

We conduct two ablation studies, one to test whether the observed brittleness generalizes beyond affine transforms, and one to control for prompt-length confounds.

7.1 Alternative Isomorphism Families

Table 5 Pass@1 (%) under alternative isomorphism families.

Data	Model	Orig	Affine	Base	Cubic
BigO	gemini-2.0-flash	65.9	50.3 (-24%)	52.2 (-21%)	52.8 (-20%)
	Codestral-22B-v0.1	22.0	12.6 (-43%)	13.4 (-39%)	11.7 (-47%)
	Qwen2.5-Coder-32B-Instruct	30.1	18.7 (-38%)	19.8 (-34%)	17.5 (-42%)
Effi	gemini-2.0-flash	54.4	29.3 (-46%)	26.9 (-51%)	37.8 (-31%)
	Codestral-22B-v0.1	21.3	10.7 (-50%)	12.5 (-41%)	9.8 (-54%)
	Qwen2.5-Coder-32B-Instruct	39.5	20.1 (-49%)	22.5 (-43%)	18.0 (-54%)
MBPP	gemini-2.0-flash	80.8	58.1 (-28%)	52.6 (-35%)	60.5 (-25%)
	Codestral-22B-v0.1	72.0	44.1 (-39%)	49.3 (-32%)	43.2 (-40%)
	Qwen2.5-Coder-32B-Instruct	84.0	45.4 (-46%)	50.5 (-40%)	43.0 (-49%)

To test whether the brittleness generalizes beyond affine transforms, we ablate with two alternative bijection families targeting distinct representational axes.

1. **Base conversion:** The decimal digit string of each integer is treated as if it were written in base b , yielding a different numeric value. For example, with $b=7$ the integer 25 has digit string "25", which read in base 7 gives $2 \times 7 + 5 = 19$, so $T(25) = 19$. The inverse recovers the original value by inverting the conversion. The base-7 representation of 19 is "25", whose digits read in base 10 give 25. This replaces the arithmetic overhead of affine inversion with a format-parsing overhead, testing whether the model can reason about numeral systems rather than polynomial algebra.
2. **Cubic polynomials:** $x' = x^3 + c$, requiring explicit cube-root inversion, a qualitatively harder challenge than a linear inverse.

We run three models across all datasets as shown in Table 5. All three families produce drops of comparable magnitude. On BigOBench, Gemini-2.0-Flash loses 21% (base) and 20% (cubic), closely tracking the 24% affine drop. On EffiBench, base conversion is steeper than affine (-51% vs. -46%), while the cubic transform is milder (-31%). On MBPP, both alternatives again bracket the affine result. The consistency across families confirms that the brittleness reflects general value-space sensitivity rather than an artifact of any particular transform.

Table 6 Pass@1 (%) under dead-code insertion controlling for the prompt-length confound. Dead-code blocks match the token-count increase of the ISO contract while leaving the task and I/O unchanged.

Data	Model	Orig	Iso	Dead Code
BigO	gemini-2.0-flash	65.9	50.3 (-24%)	65.1 (-1%)
	Codestral-22B-v0.1	22.0	12.6 (-43%)	20.5 (-7%)
	Llama-3.1-70B-Instruct	42.6	20.9 (-51%)	41.0 (-4%)
MBPP	gemini-2.0-flash	80.8	58.1 (-28%)	81.4 (+1%)
	Codestral-22B-v0.1	72.0	44.1 (-39%)	68.5 (-5%)
	Llama-3.1-70B-Instruct	71.8	41.4 (-42%)	69.5 (-3%)

7.2 Prompt Length as a Confound

The ISO condition appends an encoding contract to the prompt, increasing its length. To rule out prompt length as a confound, we insert semantically irrelevant Python code blocks that match the token-count increase of the ISO contract while leaving the task and I/O unchanged. If length alone drives the observed drops, dead-code insertion should produce comparable degradation. Table 6 shows it does not. Dead-code insertion causes negligible drops of 1 to 7% relative, while ISO degrades the same models by 24 to 51% on the same benchmarks. The gap is consistent across all model-dataset pairs. On BigOBench, gemini-2.0-flash loses 0.8 points with dead code versus 15.6 points under ISO, and Llama-3.1-70B loses 1.6 points versus 21.7. On MBPP, Gemini-2.0 under dead code actually gains 0.6 points (+1%), a fluctuation we attribute to noise. In every case, the ISO drop exceeds the dead-code drop by an order of magnitude, confirming that the performance degradation is driven by the representational load of contract compliance, not by prompt length.

8 Discussion

8.1 Why the Decoder-Side Bottleneck May Be Architectural

Our decomposition experiments indicate that decoding accounts for most of the isomorphism penalty (Section 6.3.1). A plausible reading of this is the asymmetry between prompt understanding and generation. On the encoder side, the model can integrate the contract and task into a coherent representation before producing any output [45], so it can attend to the whole prompt at once. Decoding, by contrast, is sequential, and each token depends only on the prompt and prior outputs [46]. To satisfy the encoding contract, the model has to preserve an implicit arithmetic state across many steps while also maintaining correct Python syntax and contract compliance, and this kind of constrained multi-step generation plausibly stresses working memory in ways that standard next-token training does not explicitly support [47]. The same asymmetry may also help explain why synonym fuzzing saturates at scale (Section 6.1), since lexical variation is resolved in the context-rich regime where scaled representations tend to collapse equivalent forms [48], whereas isomorphism shifts the

load onto the decoder, which must serialize a novel computation token by token. If this account is correct, the bottleneck is closer to architectural than incidental and might persist in autoregressive models even as scale narrows the gap, though our experiments speak to the size of the effect rather than to its mechanism directly.

8.2 Representational Load as a Practical Test of Generalization

The term “generalization” is highly ambiguous in the code LLM literature. At one extreme, it might require generating *genuinely novel algorithms*, that is, solutions that do not resemble any training instance in structure or strategy. Our work does not study this aspect of generalization, though some benchmarks, such as ARC-AGI [49, 50], do. At the other extreme, generalization can simply mean producing correct outputs on unseen inputs, which is trivially satisfied by any model that passes a held-out test set (qualitatively different from its training data). We study a point between these, generalization under a representational load the model could not have seen, and our results suggest that scaled models have internalized algorithmic strategies well enough to transport them across interfaces they have never encountered. This looks closer to what Chollet [51] terms *developer-aware generalization*, the ability to handle situations not anticipated by the system’s designers but lying within the span of its learned priors. We read this as a sign that the field’s methods for studying generalization may not have kept pace with the models, and it points toward evaluation frameworks that test whether a model can *apply* what it knows under novel conditions rather than only *reproduce* what it has seen.

8.3 Scope of the Metamorphic Claim

The biconditional that gives our protocol its sharpness relies on bijective value transforms over numeric test inputs and outputs. Our findings, therefore, characterize code-LLM behavior specifically on numeric programming tasks, and what they indicate there is fairly narrow, that under representational stress, scaled models appear to retain the algorithmic core and mostly fail at re-serializing the encoded interface. Other classes of representational load, such as novel string-formatting contracts, alternate data layouts, or project-local API conventions, are equally important in practice, but the strict metamorphic-testing guarantee does not directly transfer, because constructing a closed bijection on non-numeric input spaces is itself an open problem. Designing metamorphic-style oracles for those loads while preserving the same provable equivalence on the underlying task is a natural extension we leave for future work.

8.4 Solution Multiplicity Complicates the Memorization Narrative

Our opcode analysis indicates that scaled models, particularly on BigOBench and EffiBench, span a broad, multi-modal entropy distribution under ORIGINAL conditions (Section 6.3.2). We interpret these modes as distinct algorithmic families. A sorting problem might admit $O(n \log n)$ divide and conquer, $O(n^2)$ comparison-based, and $O(n)$ counting-based solutions, each with a characteristic opcode signature. Smaller

models, by contrast, tend toward narrow, often unimodal distributions, frequently collapsing to a single solution strategy per problem. If this reading holds, the multiplicity has a bearing on how memorization is interpreted, since methodologies targeting the decoder may be confounded when a model can stochastically switch between several “correct” solutions.

9 Threats to Validity

Construct validity. Static opcode entropy is a proxy for algorithmic strategy and not a direct measurement of it. Two programs with the same opcode histogram can differ in runtime behavior, and two behaviourally identical programs can compile to different bytecode, for instance, a list comprehension against an explicit loop. The Jensen-Shannon divergence we report is invariant to opcode ordering, so it tracks the compositional makeup of a solution rather than its control-flow structure, and a model could, in principle, reorganize its control flow while leaving the histogram unchanged. We mitigate these effects by reporting divergence over both passing and all generations and by reading the metric only as relative movement between matched conditions rather than as an absolute fingerprint. A second construct concern is that representational load is observable only through the transforms we instantiate, so the affine, base-conversion, and cubic families stand in for a property we cannot measure directly.

Internal validity. ISO lengthens the prompt, and our dead-code ablation (Section 7.2) bounds the share of degradation attributable to length alone at roughly 7%. We decode greedily at $T=0.0$ to strip sampling variance, which collapses pass@5 toward pass@1 and may understate the headroom a frontier model would show under sampling. The exact wording of the encode-decode contract is a further free variable, since a differently phrased instruction could prime the decoder more or less effectively, and we hold the phrasing fixed rather than searching over it. For the closed-source models, we query hosted endpoints whose system prompts and checkpoint versions are not disclosed, so a silent provider-side change would surface as a behavioral shift we could not attribute. Finally, the *Seen* labels in the CoDeC probe are probable rather than certain, as noted in Section 5.3, though the asymmetry is shared across every model and does not move the trend.

External validity. Our isomorphisms cover three integer-valued bijection families, all of our prompts are Python, and the metamorphic guarantee holds only over numeric input and output. Whether the same narrow-channel behavior appears under string-formatting contracts, structured data layouts, or project-local API conventions is untested, and constructing a closed bijection on those spaces is itself an open problem, as we discuss above. Our model panel is dense throughout and excludes mixture-of-experts architectures, where routing may interact with memorization in ways a dense model does not exhibit. The three benchmarks also favor self-contained algorithmic tasks, so our results speak to function-level synthesis rather than repository-scale or agentic coding. Broader transform families, additional languages, MoE models, and larger task units remain future work.

10 Conclusion

We do not claim that memorization is absent from scaled code LLMs, only that the diagnostics commonly used to detect it appear to stop discriminating at scale. In our experiments, synonym fuzzing and dead-code insertion on the encoder side, and CoDeC on the decoder side, lose much of their signal on frontier models, so a pass@1 drop or a likelihood shift is, on its own, weak evidence of memorization. Holding the algorithmic task fixed under an I/O isomorphism lets us separate representational load from memorization, and scaled models tend to keep the same solution families while mis-serializing the encoded interface.

The takeaway we would offer is practical. Treat a bare pass@ k drop as a starting point rather than a memorization finding, separate representational load before attributing any drop to recall, and report solution-space stability alongside correctness. For most software engineering settings the more useful question is not whether a solution was seen in training but whether a model can carry a known algorithm through an unfamiliar interface. And if memorization is to be studied in the stronger sense the term usually implies, we would suggest approaching it from a different vantage point than the surface and likelihood probes inherited from prior work, which give little to build on at current scales.

Declarations

Funding. This research was funded by the Luxembourg National Research Fund (FNR) in collaboration with the University of Luxembourg.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Data availability. The dataset generated and analysed in this study is openly available in Zenodo at <https://doi.org/10.5281/zenodo.19248561>.

Code availability. The code is openly available at https://github.com/pkrajput/isomorphic_prompts.

Author contributions. Prateek Kumar Rajput framed the methodology and wrote the manuscript. Alberick Euraste Djiré and Abdoul Aziz Bonkougou helped run the experiments. Yewei Song and Iyiola Emmanuel Olatunji contributed to reviewing and framing the manuscript. Jacques Klein and Tegawendé F. Bissyandé served as project advisors and principal investigators. All authors read and approved the final manuscript.

References

- [1] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
- [2] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021)

- [3] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770 (2023)
- [4] Zhang, L., He, S., Zhang, C., Kang, Y., Li, B., Xie, C., Wang, J., Wang, M., Huang, Y., Fu, S., et al.: Swe-bench goes live! arXiv preprint arXiv:2505.23419 (2025)
- [5] Liang, S., Garg, S., Moghaddam, R.Z.: The swe-bench illusion: When state-of-the-art llms remember instead of reason. arXiv preprint arXiv:2506.12286 (2025)
- [6] Chakraborty, S., Ahmed, T., Ding, Y., Devanbu, P.T., Ray, B.: Natgen: generative pre-training by “naturalizing” source code. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 18–30 (2022)
- [7] Wang, S., Li, Z., Qian, H., Yang, C., Wang, Z., Shang, M., Kumar, V., Tan, S., Ray, B., Bhatia, P., et al.: Recode: Robustness evaluation of code generation models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13818–13843 (2023)
- [8] Mastropaolo, A., Pascarella, L., Guglielmi, E., Ciniselli, M., Scalabrino, S., Oliveto, R., Bavota, G.: On the robustness of code generation techniques: An empirical study on github copilot. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2149–2160 (2023). IEEE
- [9] Chen, J., Zhenhao, L., Xing, H., Xin, X.: Nlperturbator: Studying the robustness of code llms to natural language variations. ACM Transactions on Software Engineering and Methodology (2024)
- [10] Djiré, A.E., Kaboré, A.K., Barr, E.T., Klein, J., Bissyandé, T.F.: Memorization or interpolation? detecting llm memorization through input perturbation analysis. arXiv preprint arXiv:2505.03019 (2025)
- [11] Euraste, D.A., Kader, K.A., Samhi, J., Barr, E.T., Klein, J., Bissyandé, T.F.: Learned or memorized? quantifying memorization advantage in code llms. arXiv preprint arXiv:2604.13997 (2026)
- [12] Riddell, M., Ni, A., Cohan, A.: Quantifying contamination in evaluating code generation capabilities of language models. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 14116–14137 (2024)
- [13] Zawalski, M., Boubdir, M., Bałazy, K., Nushi, B., Ribalta, P.: Detecting data contamination in llms via in-context learning. arXiv preprint arXiv:2510.27055 (2025)

- [14] Oren, Y., Meister, N., Chatterji, N.S., Ladhak, F., Hashimoto, T.: Proving test set contamination in black-box language models. In: The Twelfth International Conference on Learning Representations (2023)
- [15] Golchin, S., Surdeanu, M.: Data contamination quiz: A tool to detect and estimate contamination in large language models. *Transactions of the Association for Computational Linguistics* **13**, 809–830 (2025)
- [16] Nie, Y., Wang, C., Wang, K., Xu, G., Xu, G., Wang, H.: Decoding secret memorization in code llms through token-level characterization. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2880–2892 (2025). IEEE
- [17] Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al.: The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027* (2020)
- [18] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* **21**(140), 1–67 (2020)
- [19] Chen, T.Y., Cheung, S.C., Yiu, S.M.: Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology (1998)
- [20] Segura, S., Fraser, G., Sanchez, A.B., Ruiz-Cortés, A.: A survey on metamorphic testing. *IEEE Transactions on Software Engineering* **42**(9), 805–824 (2016)
- [21] Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., *et al.*: Extracting training data from large language models. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 2633–2650 (2021)
- [22] Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., Carlini, N.: Deduplicating training data makes language models better. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8424–8445 (2022)
- [23] Coignion, T., Quinton, C., Rouvoy, R.: A performance study of llm-generated code on leetcode. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, pp. 79–89 (2024)
- [24] Matton, A., Sherborne, T., Aumiller, D., Tommasone, E., Alizadeh, M., He, J., Ma, R., Voisin, M., Gilsenan-McMahon, E., Gallé, M.: On leakage of code generation evaluation datasets. In: Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 13215–13223 (2024)

- [25] Sainz, O., Campos, J., García-Ferrero, I., Etxaniz, J., Lacalle, O.L., Agirre, E.: Nlp evaluation in trouble: On the need to measure llm data contamination for each benchmark. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 10776–10787 (2023)
- [26] Shi, W., Ajith, A., Xia, M., Huang, Y., Liu, D., Blevins, T., Chen, D., Zettlemoyer, L.: Detecting pretraining data from large language models. arXiv preprint arXiv:2310.16789 (2023)
- [27] Mattern, J., Miresghallah, F., Jin, Z., Schölkopf, B., Sachan, M., Berg-Kirkpatrick, T.: Membership inference attacks against language models via neighbourhood comparison. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 11330–11343 (2023)
- [28] Zhang, C., Ippolito, D., Lee, K., Jagielski, M., Tramèr, F., Carlini, N.: Counterfactual memorization in neural language models. *Advances in Neural Information Processing Systems* **36**, 39321–39362 (2023)
- [29] Grosse, R., Bae, J., Anil, C., Elhage, N., Tamkin, A., Tajdini, A., Steiner, B., Li, D., Durmus, E., Perez, E., et al.: Studying large language model generalization with influence functions. arXiv preprint arXiv:2308.03296 (2023)
- [30] Biderman, S., Prashanth, U., Sutawika, L., Schoelkopf, H., Anthony, Q., Purohit, S., Raff, E.: Emergent and predictable memorization in large language models. *Advances in Neural Information Processing Systems* **36**, 28072–28090 (2023)
- [31] Power, A., Burda, Y., Edwards, H., Babuschkin, I., Misra, V.: Grokking: Generalization beyond overfitting on small algorithmic datasets. arXiv preprint arXiv:2201.02177 (2022)
- [32] Nanda, N., Chan, L., Lieberum, T., Smith, J., Steinhardt, J.: Progress measures for grokking via mechanistic interpretability. arXiv preprint arXiv:2301.05217 (2023)
- [33] Liu, Z., Kitouni, O., Nolte, N.S., Michaud, E., Tegmark, M., Williams, M.: Towards understanding grokking: An effective theory of representation learning. *Advances in Neural Information Processing Systems* **35**, 34651–34663 (2022)
- [34] Tirumala, K., Markosyan, A., Zettlemoyer, L., Aghajanyan, A.: Memorization without overfitting: Analyzing the training dynamics of large language models. *Advances in Neural Information Processing Systems* **35**, 38274–38290 (2022)
- [35] Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., Stoica, I.: Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974 (2024)
- [36] Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is your code generated by chatgpt

really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems* **36**, 21558–21572 (2023)

- [37] Xia, C.S., Deng, Y., Zhang, L.: Top leaderboard ranking= top coding proficiency, always? evoeval: Evolving coding benchmarks via llm. *arXiv preprint arXiv:2403.19114* (2024)
- [38] Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020)
- [39] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al.: Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022)
- [40] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., *et al.*: Language models are few-shot learners. *Advances in neural information processing systems* **33**, 1877–1901 (2020)
- [41] Huang, D., Qing, Y., Shang, W., Cui, H., Zhang, J.M.: Effibench: Benchmarking the efficiency of automatically generated code. *Advances in Neural Information Processing Systems* **37**, 11506–11544 (2024)
- [42] Chambon, P., Roziere, B., Sagot, B., Synnaeve, G.: Bigo (bench)—can llms generate code with controlled time and space complexity? *arXiv preprint arXiv:2503.15242* (2025)
- [43] Rein, D., Hou, B.L., Stickland, A.C., Petty, J., Pang, R.Y., Dirani, J., Michael, J., Bowman, S.R.: Gpqa: A graduate-level google-proof q&a benchmark. In: *First Conference on Language Modeling* (2024)
- [44] Biderman, S., Schoelkopf, H., Anthony, Q.G., Bradley, H., O’Brien, K., Hallahan, E., Khan, M.A., Purohit, S., Prashanth, U.S., Raff, E., *et al.*: Pythia: A suite for analyzing large language models across training and scaling. In: *International Conference on Machine Learning*, pp. 2397–2430 (2023). PMLR
- [45] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
- [46] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., *et al.*: Language models are unsupervised multitask learners. *OpenAI blog* **1**(8), 9 (2019)
- [47] Dziri, N., Lu, X., Sclar, M., Li, X.L., Jiang, L., Lin, B.Y., Welleck, S., West, P., Bhagavatula, C., Le Bras, R., *et al.*: Faith and fate: Limits of transformers on

- compositionality. *Advances in neural information processing systems* **36**, 70293–70332 (2023)
- [48] Ethayarajh, K.: How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 55–65 (2019)
- [49] Chollet, F., Knoop, M., Kamradt, G., Landers, B.: Arc prize 2024: Technical report. *arXiv preprint arXiv:2412.04604* (2024)
- [50] Chollet, F., Knoop, M., Kamradt, G., Landers, B., Pinkard, H.: Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831* (2025)
- [51] Chollet, F.: On the measure of intelligence. *arXiv preprint arXiv:1911.01547* (2019)